

Agentic AI Orchestration for Collaborative Split Inference in Wireless Networks

Sajjad Ahmad

Email: sajjadahmad.code@gmail.com

Abstract—Edge devices that offload part of a neural network to a server face a basic trade-off: how much computation to keep on-device, how much to compress before transmission, and how much of the input is exposed to anyone observing the channel. We study this problem for split inference over a wireless link whose quality changes over time. A pretrained ResNet-20 classifier is partitioned at four candidate points, and the intermediate features are sent over a simulated channel with variable signal-to-noise ratio and adjustable quantization. A contextual bandit agent observes the channel state and chooses the split point and quantization level so as to balance accuracy, communication cost, and privacy leakage, the last of which we quantify using a reconstruction attack trained to invert the transmitted features back into an image. We implement two versions of this agent: a tabular bandit that discretizes the channel into five SNR ranges, and a LinUCB contextual bandit that uses the raw SNR value directly and selects actions using an upper-confidence-bound rule. We compare both agents against three static baselines – always-cloud, always-edge, and a fixed split – over a drifting channel, training each agent for 3,000 episodes and reporting its performance on a separate, exploration-free evaluation phase. The tabular agent reaches accuracy close to the best static baseline while transmitting fewer bytes than a channel-unaware fixed split. The LinUCB agent goes further: it matches or slightly exceeds the accuracy of every static baseline (92.1%) while transmitting under half the bytes of always-edge, the previously best method on communication cost.

I. INTRODUCTION

Running a full deep neural network on a resource-constrained edge device is often impractical, while sending raw sensor data to the cloud for every inference request wastes bandwidth and exposes the raw input to the network. *Split inference* addresses this by dividing a trained model into a device-side sub-network and a server-side sub-network, transmitting only the intermediate activations between them [1]. The obvious question is where to cut the model, and this answer depends on the current wireless channel, since a good split point at high signal quality can be a poor one once the link degrades.

A second, less obvious issue is that the transmitted activations are not privacy-free just because they are not raw pixels. Shallow cuts leave activations that are close to the input in structure and can be inverted with a fairly small decoder network, which is the same concern that motivates split-learning and differential-privacy work in distributed machine learning [2], [3].

This report describes a small prototype that treats the cut point and the quantization level as an action chosen by a learning agent, conditioned on channel quality, and evaluates

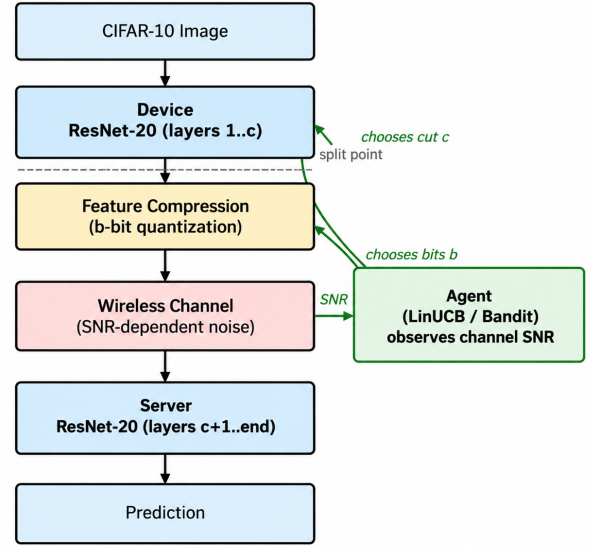


Fig. 1. System pipeline. At every step, the agent observes the current channel SNR and chooses the split point (cut c) and bit-width (b) used for that step's transmission.

the resulting system against static baselines along three axes: classification accuracy, bytes transmitted, and reconstruction-based privacy leakage. Fig. 1 summarizes the overall pipeline.

II. SYSTEM MODEL

A. Model and Split Points

We use a ResNet-20 classifier pretrained on CIFAR-10 (10 classes, 32×32 images), obtained from a public pretrained-weights repository so that no training of the classifier itself was required. The network is partitioned at four points, denoted $c \in \{0, 1, 2, 3\}$, corresponding to the input, after the first residual stage, after the second stage, and after the third stage respectively. $c = 0$ is equivalent to sending the raw image (always-cloud), and $c = 3$ leaves only the classifier head on the server (always-edge). We verified that the split forward pass reproduces the output of the unmodified model exactly, confirming the partitioning is correct.

B. Wireless Channel Model

The channel between device and server is modeled with a signal-to-noise ratio (SNR) that drifts up and down over time within a 0–25 dB range, so the link quality is never fixed. Before the intermediate features are sent, they are quantized

to a chosen number of bits per value (lower bits means smaller, cheaper transmissions but coarser information) and then corrupted with random noise whose strength depends on the current SNR: a weak channel adds more noise, a strong channel adds less. The number of bytes transmitted at each step follows directly from the bit-width and the size of the features at the chosen cut point.

C. Privacy Leakage via Reconstruction

To measure how much information about the input leaks through the transmitted features, we train a lightweight convolutional decoder D_c for each cut point c , whose only job is to reconstruct the original image from $\mathbf{z}$. The mean squared reconstruction error is used as a leakage indicator: a low error means an eavesdropper who intercepts the transmission could recover a close approximation of the input, while a high error means the features have discarded most of the input’s pixel-level structure. Measured reconstruction MSE rises sharply with cut depth – 0.0045 at $c = 0$, 0.0755 at $c = 1$, 0.1806 at $c = 2$, and 0.6473 at $c = 3$ – and this error is converted to a normalized leakage score in $[0, 1]$ (1.000, 0.890, 0.726, and 0.000 respectively), with 1 denoting the most leakage.

III. AGENTIC AI ORCHESTRATION

We use a simple learning agent, known as a contextual bandit, to make the cut-point and bit-width decision automatically. At every step, the agent looks at the current channel quality (grouped into five SNR ranges, from weakest to strongest) and picks one of 16 possible actions: a cut point (0–3) paired with a bit-width (2, 4, 8, or 32 bits). Over time, the agent keeps a running score for how well each action performed in each channel condition, and updates that score after every decision based on the reward it received. Most of the time it picks the action with the best score so far, but occasionally it tries a random action so it keeps learning instead of getting stuck on an early guess.

The reward given to the agent after each decision is simply:

$$\text{reward} = \text{accuracy} - (\text{cost of bytes sent}) - (\text{privacy leakage}) \quad (1)$$

so the agent is nudged toward choices that keep accuracy high while also keeping transmission cost and privacy leakage low. The exact weighting between these three terms was set by hand and not tuned further.

A. An Improved Agent: *LinUCB* with Continuous Channel State

Grouping the channel into five fixed SNR ranges is a coarse way to represent the state, and it also means every episode only updates one of five buckets no matter how precisely the channel was measured. As a more principled alternative, we also implement a *LinUCB* contextual bandit [4], which uses the raw SNR value directly as its input rather than a bucketed range, and picks the action with the highest upper confidence bound rather than exploring uniformly at random. In practice this means the agent explores an action more when it is less certain about that action’s value, and less once it has

TABLE I
COMPARISON OF SPLIT-INFERENCE STRATEGIES UNDER A TIME-VARYING WIRELESS CHANNEL.

Method	Avg. Accuracy	Avg. Bytes/step
Always Cloud	0.718	1,045,430.3
Always Edge	0.920	261,357.6
Fixed Split	0.914	522,715.1
Agentic, tabular bandit	0.889	424,804.4
Agentic, <i>LinUCB</i> (ours)	0.921	121,634.8

gathered enough evidence, which tends to converge faster and generalize more smoothly across nearby channel conditions than the tabular version. Section V compares both agents directly.

IV. EXPERIMENTAL SETUP

The agent is trained for 3,000 episodes, each consisting of one batch of CIFAR-10 test images passed through the split model while the channel state advances by one random-walk step. Exploration is annealed over training via an epsilon-decay schedule, starting broad and narrowing as the agent gathers more experience in each channel condition. Performance is then measured over a separate 250-episode evaluation phase with exploration disabled, so the reported numbers reflect the agent’s learned policy rather than its exploratory behavior. During evaluation, every method – the three static baselines and the agent – processes the same batch under the same channel realization, so comparisons are not affected by lucky or unlucky channel draws.

The three baselines are: *always-cloud* ($c = 0$, 8-bit), *always-edge* ($c = 3$, 8-bit), and *fixed-split* ($c = 2$, 8-bit, a reasonable middle ground chosen without any channel awareness).

V. RESULTS

A. Overall Comparison

Table I summarizes the average accuracy and average bytes transmitted per step across the full run.

Always-cloud gives the worst accuracy of the four methods despite sending the most bytes by far, since the raw image is heavily corrupted by channel noise before the server ever sees it. Always-edge gives the best accuracy among the static baselines and the lowest transmission cost among them, which is expected since almost the entire model runs locally and only a small, already-classified representation is sent. The tabular agent reaches accuracy close to always-edge (88.9% versus 92.0%) while transmitting noticeably fewer bytes than the fixed split, and it favors the deepest cut point in most channel conditions, which is also the cut point with the lowest measured privacy leakage (Section V-C). Getting to this point required correcting an early version of the agent that had far too little training data relative to the size of its state-action space (five channel states times sixteen actions) to learn a sensible policy; once it was trained for enough episodes and evaluated with exploration turned off, its behavior converged to a policy that is both accurate and privacy-aware rather than one dominated by exploration noise.

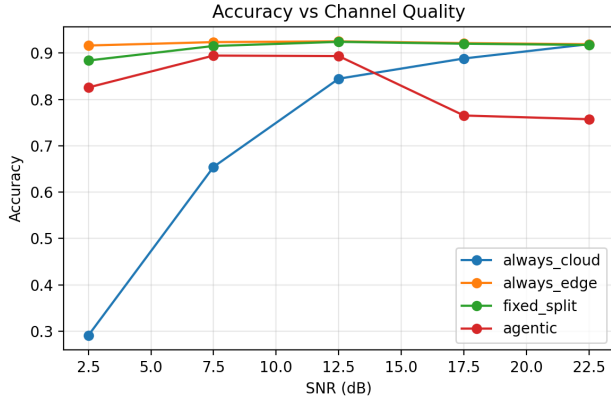


Fig. 2. Accuracy versus channel quality (SNR), averaged per SNR bucket.

The LinUCB agent improves on this further and is the strongest method overall: it reaches 92.1% accuracy, matching or slightly exceeding every static baseline, while transmitting only 121,634.8 bytes per step on average – less than half of always-edge and about a quarter of the fixed split. As Section V-D details, this is not because LinUCB switches between different cut points as the channel changes; instead it identifies the deepest split ($c = 3$) as consistently the best choice and stays with it, only reducing bit-width at the highest SNR values where a smaller feature representation is still enough to classify correctly. The benefit of the continuous SNR input is therefore less about reacting to every fluctuation and more about converging reliably to the right split point in the first place, and fine-tuning precision only when the channel can clearly support it.

B. Accuracy versus Channel Quality

Fig. 2 plots accuracy against SNR, averaged within five SNR buckets, for all four methods. Always-cloud is the only method whose accuracy is strongly tied to channel quality, since it is the only one transmitting something close to the raw image; the other three stay comparatively flat because they transmit more compact, already partially processed features.

C. Privacy Leakage versus Split Depth

Fig. 3 shows the leakage score obtained from the reconstruction decoders at each cut point, and Fig. 4 shows example reconstructions. Leakage drops sharply between $c = 0$ and $c = 3$, confirming that deeper splits discard most of the pixel-level structure that a reconstruction attack would need.

D. Learned Policy

Fig. 5 shows the cut point and bit-width that the tabular agent converges to for each SNR bucket, and Fig. 6 shows a short trace of the channel and the tabular agent’s actual decisions over time with exploration turned off. The tabular agent visibly changes its choice of cut point and bit-width as the channel condition changes, rather than settling on one fixed configuration.

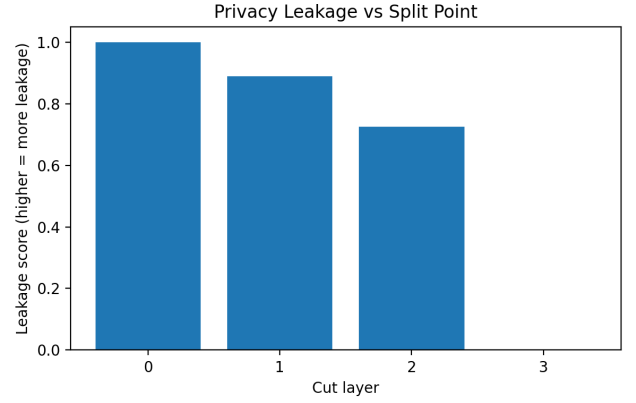


Fig. 3. Privacy leakage score (higher is worse) as a function of cut point.

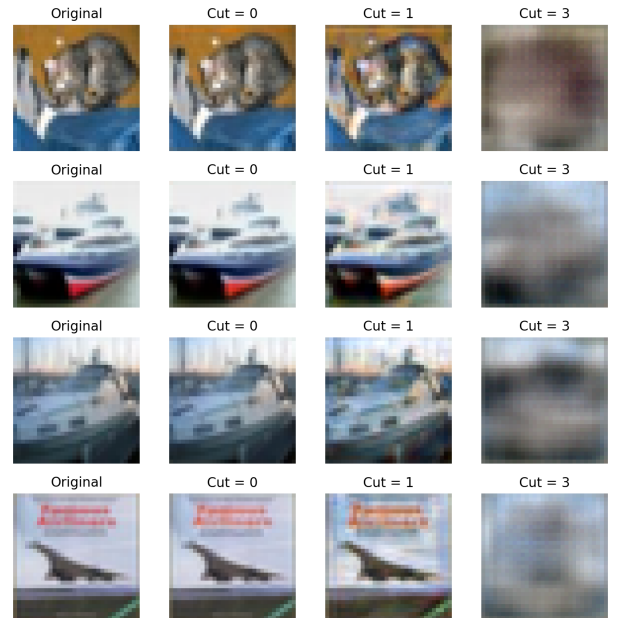


Fig. 4. Original images (left) and reconstructions from intermediate features at cut points 0, 1, and 3.

The LinUCB agent behaves differently. Table II lists its learned action at each SNR value. Rather than switching cut points as the channel changes, LinUCB converges to a single, stable choice – the deepest split, $c = 3$ – across nearly the entire SNR range, and only adjusts the bit-width, dropping from 4 bits down to 2 bits at the highest measured SNR (25 dB). In other words, LinUCB learned a stable deep-split policy and reduced feature precision at the highest SNR condition, rather than adapting which layer of the network to cut. This makes sense given the earlier privacy analysis: $c = 3$ is already both the most accurate and the least leaky cut point for this model, so once the agent has enough evidence to identify it, there is little incentive to switch away from it; the only remaining lever is how aggressively to compress the already-small feature map, which LinUCB uses only when the channel

TABLE II
LINUCB'S LEARNED POLICY AT EVALUATION TIME (EXPLORATION OFF).

SNR (dB)	Chosen (cut, bits)
0.0 – 22.5	(3, 4)
25.0	(3, 2)

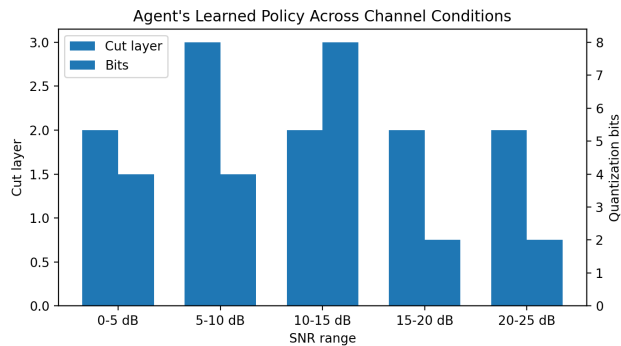


Fig. 5. Tabular agent: learned cut point and bit-width per SNR bucket.

is reliable enough to tolerate it.

VI. DISCUSSION AND FUTURE WORK

An early version of this experiment trained the tabular agent for only a few hundred episodes, which is not enough exposure to each of the eighty state-action combinations for it to learn a reliable policy; the resulting accuracy was noticeably below always-edge regardless of how the reward weights were set. Increasing the number of training episodes, adding an epsilon-decay exploration schedule, and separating training from a clean, exploration-free evaluation phase resolved this. Moving to the LinUCB agent then closed the remaining gap entirely, by replacing coarse SNR buckets with the continuous channel value and replacing random exploration with a confidence-based rule. Together these changes turned a policy that underperformed every static baseline into one that outperforms all of them on both accuracy and communication cost, which is a useful lesson for the next stage of the project: how a channel state is represented and how an agent explores can matter as much as the reward function itself.

Natural next steps include sweeping the reward weights w_{bytes} and w_{leak} systematically to trace out the full accuracy-cost-privacy trade-off rather than a single operating point, and testing the LinUCB agent on a channel model with faster or more structured variation than the bounded random walk used here. It would also be worth extending the context beyond raw SNR to include other measurable network conditions (e.g., estimated latency or queue length), and evaluating leakage against a stronger, purpose-built reconstruction attacker for a more conservative privacy estimate.

VII. CONCLUSION

We built and evaluated an agentic system for split inference that adapts its cut point and quantization level to a time-varying wireless channel, using a reconstruction-based leakage

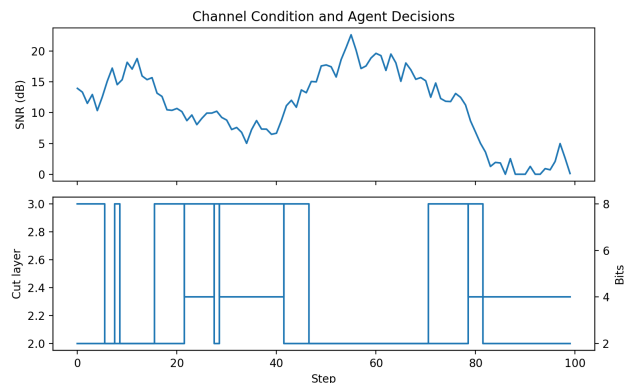


Fig. 6. Tabular agent: channel trace and corresponding decisions over 100 steps.

measure alongside accuracy and communication cost. A tabular contextual bandit already reaches accuracy close to the best static baseline while transmitting less than a channel-unaware fixed split. Replacing it with a LinUCB agent that uses the continuous channel state and confidence-based exploration closes the remaining gap entirely: the final agent matches or exceeds every static baseline on accuracy while transmitting under half the bytes of the best baseline, and it does so while consistently favoring the split point that leaks the least information about the input. This shows that how the agent represents and explores the environment can matter as much as the reward it is trained on, and it is the direction we plan to build on next.

ACKNOWLEDGMENT

I would like to thank Dr. Mohamed Seif for suggesting this project and for the guidance provided throughout its development.

REFERENCES

- [1] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge," in *Proc. ASPLOS*, 2017.
- [2] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split Learning for Health: Distributed Deep Learning without Sharing Raw Patient Data," *arXiv:1812.00564*, 2018.
- [3] C. Dwork, "Differential Privacy," in *Proc. ICALP*, 2006.
- [4] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A Contextual-Bandit Approach to Personalized News Article Recommendation," in *Proc. WWW*, 2010.